Using Blocks

YMPrint uses a special "block syntax" for defining custom content types. Some of these are similar to content types you might expect to see in markdown, e.g. images or code blocks, or extensions to common markdown such as admonitions.

Images (`_img`)

Figure 1: The catpuccin cat

Admonitions

- The following admonitions blocks are available:
  - _info
  - _warning
  - _danger
  - _tip
  - _note
- They are rendered below

> **INFO**
>
> Here is an "info" admonition

> **WARNING**
>
> Here is a "warning" admonition

✖ DANGER

Here is a danger admonition

✔ TIP

Here is a helpful tip

✎ NOTE

This is useful when you want to give a note

Block quotes (`_blockquote`)

> *“What you do makes a difference, and you have to decide what kind of difference you want to make.”*
>
> — Jane Goodall

Page breaks (`_pagebreak`)

You can insert an arbitrary page break using _pagebreak, such as the one below. _pagebreak accepts null as a value.

Horizontal rules (`_hrule`)

Unlike a horizontal rule in Markdown (which does not have properties you can adjust) the `_hrule` block in YMPrint has optional parameters like width_ratio, thickness, cap, and color that you can manipulate.

Vertical spacers (`_spacer`)

You can create arbitrary amounts of vertical white space so you can manually adjust the positioning of your content.

The `_spacer` object accepts a distance in points for the amount of vertical space to insert.

Below, a spacer is inserted in between each line of text

There is a 5 pt spacer above

There is a 10 pt spacer above

There is a 20 pt spacer above

There is a 40 pt spacer above

Executable code Blocks (`_py`)

In a `_py` code block, Python code will be executed using `exec()` with the "vars" dictionary from the document context passed in as its global scope and an optional namespace dictionary as its local scope. If no namespace is passed, the "vars" dictionary will be the local scope.

Any "_vars" you set in your document front matter will be available to your Python code and any variables you create in your Python code will be available to your document vars.

The Python environment used to execute is the same environment that YMPrint is running insert. At this time, external Python subprocesses have not been implemented.

Once this code executes, the variables will be accessible from the "vars" key in the document context under a namespace called py1.

```python
import math
a = 3
b = 4
c = math.sin(a/b)
acc = []
for k in [a, b, c]:
    acc.append(k)
```

Now, the values of the variables a, b, and c can be included in the document:

- a = 3 (using py1.a)
- b = 4 (using py1.b)
- c = 0.6816387600233341 (using py1.c)

```
yaml_data: is being shown
you can put: any yaml data together
more examples:
  - - cell-padding
    - cell_size
    - row_size
```

Dynamic JSON variables (`_loadjson`)

You can dynamically build your vars context by including a _loadjson block in your document.

Similar to the _py block, you can pass an optional namespace parameter to load the variables into a namespace under the vars dictionary. If no namespace is passed, then the variables will be available under the vars dictionary.

Here are the values of the vars contained within the 'extravars' namespace:

- bn = 43
- dx = 0.0012